



Kinetic Audit Report Finding

Version 1.0

Vkgoud

April 30, 2025

Audit Report

Vk goud

April 13, 2025

Prepared by: Vkgoud

Lead Auditors:

- Vamshi Krishna Goud

Table of Contents

- Table of Contents
- Protocol Summary
 - Core Features
 - Advanced Features
- Risk Classification
- Audit Details
 - Scope
 - Compatibilities
- Deploy core contracts
- LOW
 - [L] - Front-running in Staking Operations

Protocol Summary

Kinetiq is a Liquid Staking protocol building on Hyperliquid. It allows users to stake their HYPE tokens and receive kHYPE tokens in return, enabling them to participate in network security while maintaining liquidity.

Core Features

Deposit HYPE tokens to mint and receive kHYPE tokens
Liquid staking allows users to utilize their staked assets
Oracle-based performance tracking and reward distribution
Upgradeable smart contracts
Role-based access control

Advanced Features

Sophisticated validator management system
Performance-based stake allocation
Emergency withdrawal system with queue management
Historical performance tracking
Gas-optimized rebalancing mechanism

Risk Classification

LOW

Audit Details

Scope

The following roles and contracts were reviewed:

- `DEFAULT_ADMIN_ROLE`
 - `OPERATOR_ROLE`
 - `MANAGER_ROLE`
 - `TREASURY_ROLE`
 - `SENTINEL_ROLE`

Compatibilities

Hyperliquid Solidity

Deploy core contracts

```
forge script script/DeployCore.s.sol
-sig "run(string)" config/testnet.json
-rpc-url $RPC_URL
-broadcast
-verify
```

LOW

[L] - Front-running in Staking Operations

Description: The `stake` function is vulnerable to front-running attacks due to the lack of slippage protection:

Impact:

An attacker could:

Monitor the mempool for staking transactions Front-run with a larger stake to reach the staking limit
Cause the original transaction to fail

Proof of Concept:

```
1 function stake() public payable nonReentrant whenNotPaused
  whenStakingNotPaused {
2     // Check whitelist if enabled
3     if (whitelistEnabled) {
4         require(isWhitelisted(msg.sender), "Address not whitelisted");
5     }
6
7     // Validate stake amount
8     require(msg.value >= minStakeAmount, "Stake amount below minimum");
9     if (maxStakeAmount > 0) {
10        require(msg.value <= maxStakeAmount, "Stake amount above
            maximum");
11    }
12    if (stakingLimit > 0) {
13        uint256 rewardsAmount = validatorManager.totalRewards();
14        uint256 netStaked = totalStaked + rewardsAmount - totalClaimed;
15        require(netStaked + msg.value <= stakingLimit, "Staking limit
            reached");
16    }
17
18    totalStaked += msg.value;
19    - uint256 kHYPEAmount = stakingAccountant.HYPEToKHYPE(msg.value);
```

```
20     kHYPE.mint(msg.sender, kHYPEAmount);
21     // ...
22 }
```

Recommended Mitigation:

Implemented a commit-reveal scheme or by add slippage protection:

```
1  function stake(uint256 minKHYPEAmount) public payable nonReentrant
   whenNotPaused whenStakingNotPaused {
2      // ... existing checks ...
3
4  +   uint256 kHYPEAmount = stakingAccountant.HYPEToKHYPE(msg.value);
5  +   require(kHYPEAmount >= minKHYPEAmount, "Slippage too high");
6
7      totalStaked += msg.value;
8      kHYPE.mint(msg.sender, kHYPEAmount);
9      // ...
10 }
```